FINAL PROJECT REPORT • Computer Vision, Image Forensics

Copy-Move Forgery Detection using SIFT and RANSAC

A classical computer-vision pipeline evaluated on the CoMoFoD dataset, with an interactive Streamlit web interface

Name	Registration No.	Contribution
Rishav Ranjan	20233345	Dataset preparation, preprocessing, Frontend Design and Integration
Manu Singhal	20233533	SIFT feature matching and RANSAC
Rishabh Jain	20233232	Evaluation, experiments, and visualizations
Nakul Tiwari	20233194	Testing, report, and presentation

Abstract

Copy-move forgery copies a region of an image and pastes it elsewhere in the *same* image to hide or duplicate content. This project implements a training-free detector for it. SIFT keypoints are matched against the image itself with a rule that forbids matches between nearby points, the matches are filtered by Lowe's ratio test, and **RANSAC** estimates a similarity transform (translation, rotation, uniform scale) that explains the duplicated area. If enough matches agree on one transform the image is flagged, and a mask is built from the convex hulls of the matching points. The system has a detector (`detect.py`), an evaluation script for CoMoFoD (`evaluate.py`) and a Streamlit web application (`app.py`) that shows the input and the highlighted output side by side. A demonstration on a JPEG-compressed CoMoFoD image was flagged with 257 matches agreeing on one transform. The detector is an educational baseline: it highlights *suspected* regions and cannot prove forgery.

1. Introduction

Photographs are trusted as evidence in journalism, law and science, yet are easy to edit. In a copy-move forgery the pasted content shares the lighting, noise and camera traits of the rest of the image, so the only reliable clue is that two different places look almost identical. The copy may also be resized, rotated or recompressed, so matching must tolerate such changes.

Problem. Given one image, decide whether a region has been duplicated within it and mark the suspected source and destination. **Objectives:**

- Implement SIFT plus RANSAC copy-move detection with geometric verification.
- Prepare CoMoFoD (forged, original and mask files) and measure image-level and pixel-level performance.
- Build a web interface to upload or paste an image and view input and detected output side by side.
- Review the baseline, record its limitations and propose measurable improvements.

Background. Block-based methods (DCT, Zernike moments) compare every image block and cover smooth areas but are slow and often fragile to rotation and scaling. Keypoint-based methods (this project; Amerini et al., 2011) are fast and recover the transform, but miss clones in textureless areas. **SIFT** gives scale- and rotation-invariant 128-value descriptors; **Lowe's ratio test** keeps a match only if the best distance $d_1 < 0.75 \cdot d_2$ (second best); **RANSAC** fits a transform to random minimal samples and keeps the one with the most agreeing matches (inliers).

2. Dataset and Preprocessing

CoMoFoD (Tralic et al., 2013) is a benchmark for copy-move detection. As described by its authors it has 260 forged image sets (200 small, 512 × 512 px; 60 large) with the original, the forged image and masks of the manipulated region, plus post-processed copies (JPEG compression, blur, noise, colour reduction, brightness and contrast changes) for robustness tests.

File	Meaning	Use in our code
`001_F.png`	Forged image	Positive sample
`001_O.png`	Original, untouched image	Negative sample
`001_B.png`	Binary ground-truth mask	Pixel metrics
`001_F_JC4.jpg` etc.	Post-processed variants (e.g. JPEG)	Skipped by evaluate.py; used for manual tests in the web app

Subset selection. `evaluate.py` walks the folder and matches names with the regular expression `^(\d+)_([FOB])$`, so only base files qualify and post-processed names are ignored. Files are grouped by ID and only IDs with both F and O are eligible. Evenly spaced IDs are chosen with `numpy.linspace` (default 20); `--ids 001,045,120` gives a fixed, reproducible list for tuning.

Preprocessing step	Where	Purpose
Read colour images (BGR); decode uploads with imdecode; convert pasted PIL images	evaluate.py, app.py	One input format for the detector
Convert to grayscale	detect.py	SIFT works on intensity
Read masks as grayscale, binarise at 127, resize with nearest neighbour	evaluate.py, app.py	Strictly 0/255 masks that match the image size
Limit longest side (default 1600 px)	app.py	Keeps matching fast for large uploads
Skip unreadable files with a message	evaluate.py	A damaged file does not stop a batch

3. System Design

One detector module is shared by the batch evaluation and the web interface, so experiment numbers and user results come from exactly the same code. Stack: Python 3, OpenCV (SIFT, BFMatcher, `estimateAffinePartial2D`, hull, morphology), NumPy, Streamlit and the optional `streamlit-paste-button`.

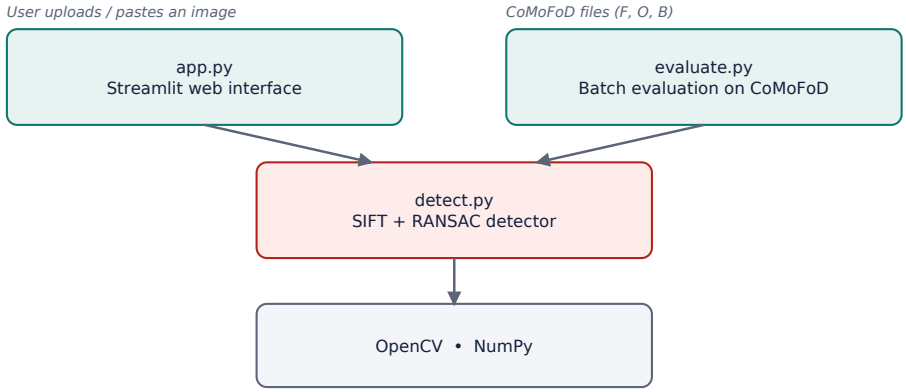


Figure 1. Module architecture: both entry points call the same detector. Project folder: `detect.py`, `evaluate.py`, `app.py`, `.streamlit/config.toml`, `data/` (CoMoFoD files) and `outputs/` (results).

4. Methodology

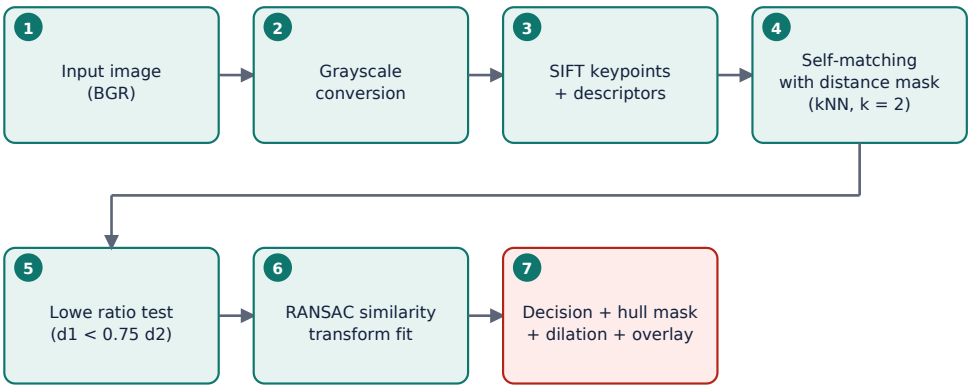


Figure 2. Detection pipeline implemented in `detect_copy_move()`.

Steps 1-3: SIFT. The image is converted to grayscale and SIFT is created with `nfeatures=2000` (strongest 2000 keypoints) and `contrastThreshold=0.02` (default 0.04) so that low-contrast areas, where many clones lie, still give keypoints. With fewer than 8 keypoints the function returns early with "Too few SIFT features".

Step 4: self-matching with a spatial constraint. Matching an image to itself would make every keypoint match itself or a neighbour. `make_spatial_match_mask()` builds an $n \times n$ matrix where entry $[i, j]$ is 0 if the two keypoints are closer than `min_distance` (50 px) and 255 otherwise (diagonal always 0). It is passed as the mask of `BFMatcher.knnMatch(k=2)`, so forbidden pairs are removed *before* the ratio test and the runner-up is also a valid distant keypoint.

Step 5: ratio test. A match is kept if m.distance < 0.75 · n.distance; self-matches are also dropped. Fewer than 3 survivors gives "Too few candidate matches".

Step 6: RANSAC. The surviving point pairs (p_i, q_i) are fitted with `cv2.estimateAffinePartial2D` to the model $q = s \cdot R(\theta) \cdot p + t$: uniform scale s, rotation θ and translation t (four degrees of freedom, two point pairs per sample), which is what a forger can do with copy, resize, rotate and paste. A match is an inlier if $\|Mp_i - q_i\| < 5$ px. Settings: 3000 iterations, confidence 0.99, 10 refinement iterations. The matrix gives $s = \sqrt{(a^2 + b^2)}$ and θ = atan2(b, a), which the web app displays.

Step 7: decision and mask. The image is flagged when inliers ≥ `min_inliers` (default 8); the inlier count acts as confidence. The convex hulls of the source and destination inlier points are filled into one mask, dilated with an elliptical kernel (17 px), tinted red (opacity 0.42) and boxed by `make_overlay()`.

Parameter	Default	Controls	Raising it usually
nfeatures	2000	Max SIFT keypoints	finds weak-texture clones; slower, more memory
min_distance	50 px	Min separation of matched points	ignores near look-alikes; may miss close clones
ratio	0.75	Ratio-test threshold	keeps more matches, so more noise
ransac_threshold	5 px	Allowed distance from the transform	tolerates rotation/scale; admits false inliers
min_inliers	8	Inliers needed to flag	fewer false alarms; may miss small clones
dilation_size	17 px	Padding of the hull mask	higher recall, lower precision

Cost. Brute-force matching is O(n^2·128); the spatial mask uses one byte per pair (about 4 MB for 2000 keypoints, 25 MB for 5000), which is fast for 512 × 512 images.

5. Implementation and Evaluation

detect.py contains `make_spatial_match_mask`, `convex_hull_or_points`, `compute_metrics` (pixel precision, recall, F1, IoU), `detect_copy_move` (returns a dictionary with decision, reason, counts, mask, inlier matches, keypoints and affine matrix, with four early-exit paths that each give a reason), `make_overlay`, and a command-line interface (`run_one`) that saves the mask, overlay, matches image and JSON. **evaluate.py** runs the detector on the F and O image of every selected ID and writes per-image masks and overlays, `per_image_results.csv` (keypoints, candidates, inliers, labels, pixel metrics) and `summary.json`.

```
python detect.py data/001_F.png --output outputs/demo --mask data/001_B.png
python evaluate.py data --ids 001,010,020,030,040 --ratio 0.7 --min-inliers 12 --output outputs/run2
```

Metrics. Image level: a flagged forged image is TP, an unflagged one FN, a flagged original FP, an unflagged original TN; accuracy = (TP+TN)/all, precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = 2TP/(2TP+FP+FN). Pixel level: precision = TP/(TP+FP), recall = TP/(TP+FN), IoU = TP/(TP+FP+FN), summed over all images (micro average); flagged pixels on originals count as false positives.

6. Web Interface (Frontend)

The interface lets a non-programmer add an image, press one button and compare input and result at once. Light, report-style design: pale grey page, white panels, navy text, teal accent; red is reserved for forgery results.

#	User action	What the app does
1	Upload a file or paste from the clipboard	Decodes to BGR, optionally limits size
2	(Optional) adjust sidebar sliders	One slider per detector parameter, defaults equal to detect.py
3	Press Detect forgery	Runs detect_copy_move in a spinner; errors shown as messages
4	Read verdict banner and counts	Red = suspected forgery, green = none flagged; keypoints, candidates, inliers
5	Compare input and output	Left: input. Right: red-tinted regions with boxes, plus download
6	Open the tabs	Predicted mask; matched features; scale, rotation, shift and JSON; score against a _B mask

Safeguards. A hash of the image is stored with each result so it is never shown beside a different image, and the negative verdict says that it does not prove authenticity. **Readability fix.** In testing, Streamlit's dark mode drew light text on the app's light background, making sidebar labels, radio options, tabs and the download button almost unreadable. We added a `.streamlit/config.toml` that forces the light theme and teal colour, and explicit dark-text CSS for every widget in `app.py`.

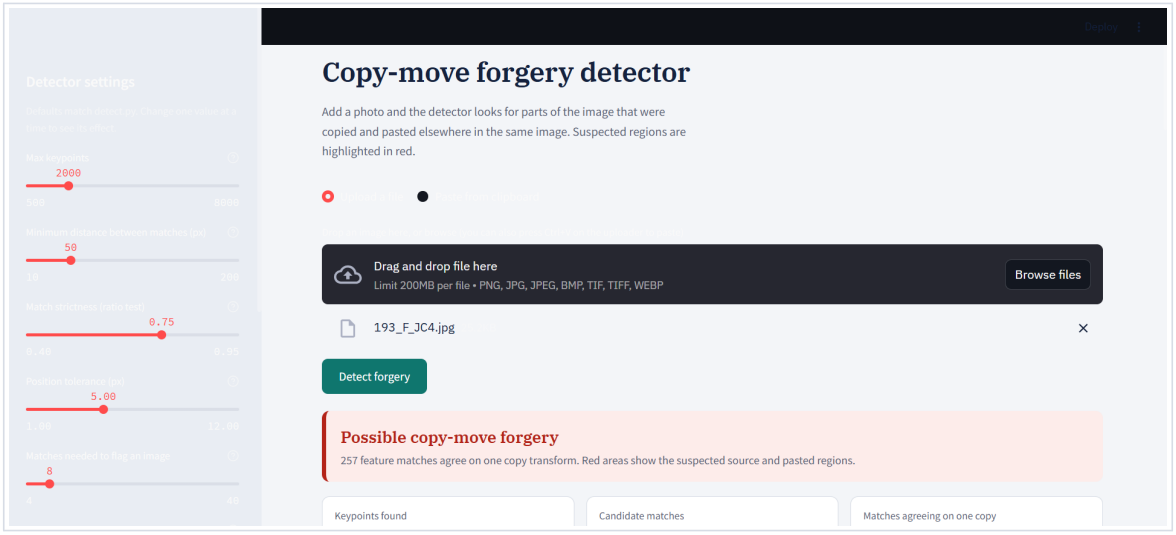


Figure 3. Early version of the interface under Streamlit's dark theme: verdict banner and counts work, but sidebar text, radio options and some controls have too little contrast. Corrected afterwards.

7. Experiments and Results

Demonstration. The file `193_F_JC4.jpg` (ID 193, forged, JPEG-compressed variant, 512 × 512 px) was uploaded with default parameters. The detector reported **"Possible copy-move forgery"** with **257** matches agreeing on one transform, far above the threshold of 8. The red regions mark two similar balcony structures with an awning and windows, consistent with a duplicated balcony, and detection survived the recompression (Figure 4). This was one manual test, not scored against the ground-truth mask.

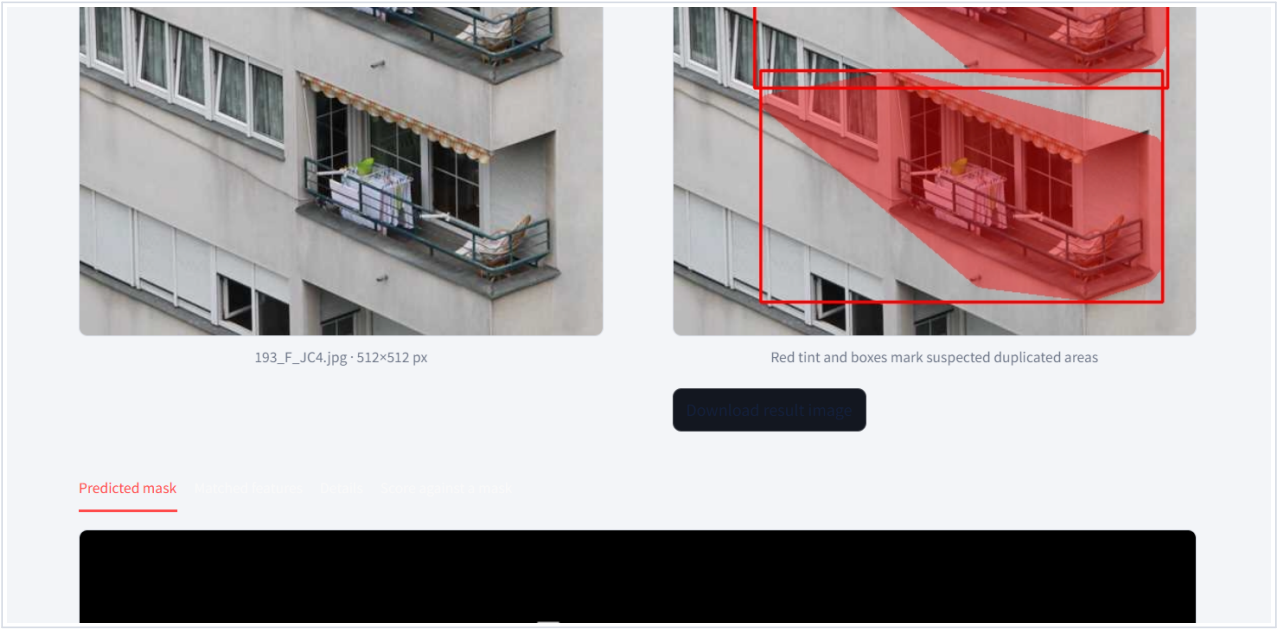


Figure 4. Web interface output for 193_F_JC4.jpg. Left: input. Right: detected regions (red tint and bounding boxes).

Quantitative results. Enter the values from `summary.json` of `evaluate.py` on the held-out test IDs (tuning is done on separate development IDs):

Metric (default parameters)	Value	Metric	Value
Forged detected (TP) / missed (FN)	*to be filled*	Image accuracy / precision	*to be filled*
Originals flagged (FP) / not flagged (TN)	*to be filled*	Image recall / F1	*to be filled*
Aggregate pixel precision / recall	*to be filled*	Aggregate pixel F1 / IoU	*to be filled*

Observations. (1) The mask is generous: the hull joins the outermost inliers and the 17 px dilation adds padding, so recall is likely high and pixel precision lower. (2) Repeating architecture (windows, balconies, tiles) can create consistent matches in untouched images, the main expected source of false alarms. (3) The mask marks both source and destination; if the dataset's B mask marks only the pasted region, pixel precision is capped near 50% by design.

8. Testing

ID	Test case	Expected behaviour	Status
T1	Upload forged image 193_F_JC4.jpg and press Detect	Verdict, side-by-side images and counts shown	Passed (Fig. 4)
T2	View interface in Streamlit dark mode	All text readable	Defect found, fixed
T3	Original (_O) image	Not flagged or low inlier count	To be recorded
T4	Forged (_F) image with its _B mask	Flagged; pixel metrics computed	To be recorded
T5	JPEG, blur, noise, brightness variants	Gradual drop, no crashes	To be recorded
T6	Corrupted file, or textureless image	Clear error / "Too few SIFT features"	To be recorded
T7	Very large image; mask of different size	Resized per sidebar; mask resized, metrics computed	To be recorded
T8	evaluate.py with a missing ID in --ids	ValueError naming the ID	To be recorded

9. Review Findings and Proposed Improvements

Found in code review; proposed work, not part of the submitted code unless integrated.

#	Finding	Proposed fix
1	Symmetric matches (A to B and B to A): RANSAC fits one direction, so about half the true matches are outliers.	Count a match as inlier if it fits the transform or its inverse.
2	Only one transform is estimated; images with several clones are partly detected.	Iterative RANSAC: fit, remove inliers, repeat.
3	Ratio test rejects regions copied more than once (best is close to second best).	Generalised 2-NN (g2NN) with k = 5.
4	Weak false-positive control on repetitive textures.	Check scale and inlier area; tune min_inliers (10 to 15).
5	Mask is a keypoint hull with fixed dilation; limits pixel IoU.	Warp with the transform and keep pixels that match their counterpart.
6	Plain SIFT; only 2000 keypoints.	RootSIFT; 5000 keypoints, lower contrast threshold, optional CLAHE.
7	evaluate.py reads each file three times in find_base_files (duplicated block).	Remove the duplicates.
8	Files keyed only by ID, so equal IDs in several folders overwrite each other.	Key by folder plus ID.
9	A forged image counts as detected even if flagged in the wrong place; micro metrics only.	Require overlap with ground truth; add macro averages.
10	B-mask meaning unverified; no tuning/test split; variants ignored.	Inspect B masks; dev/test split; robustness table.

10. Limitations

Clones in smooth regions (sky, walls, water) give few keypoints and are often missed. Localisation is approximate (sparse keypoint hulls). Genuinely repeated objects can be flagged in authentic images. Only one similarity transform is modelled. Strong blur, noise or compression removes the detail SIFT needs. The output is a lead for a human examiner, and a negative result is not proof of authenticity.

11. Conclusion and Future Work

We built an end-to-end copy-move detector: SIFT, spatially constrained self-matching, ratio test and RANSAC; a CoMoFoD evaluation script with image- and pixel-level metrics; and a Streamlit interface with side-by-side output, parameters, diagnostic views and downloads. A JPEG-compressed demo image was flagged with 257 matches agreeing on one transform. Future work: integrate the improvements in Section 9, add a dense block-based or PatchMatch detector for smooth-area clones, run the full evaluation with a dev/test split and robustness table, add a batch mode to the interface, and compare with learned detectors.

12. Team Contributions

Member (Reg. No.)	Contribution	Related report parts
Rishav Ranjan (20233345)	Dataset preparation, preprocessing, Frontend Design and Integration	Section 2; Section 6 (app.py, theme file, readability fix)
Manu Singhal (20233533)	SIFT feature matching and RANSAC	Section 4 (detect.py core)
Rishabh Jain (20233232)	Evaluation, experiments, and visualizations	Sections 5 and 7 (evaluate.py, metrics, overlays)
Nakul Tiwari (20233194)	Testing, report, and presentation	Section 8; this report

Usage

```
pip install streamlit opencv-contrib-python numpy streamlit-paste-button   # last one optional
streamlit run app.py      # keep app.py with detect.py; .streamlit/config.toml next to app.py
```

References

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *IJCV*, vol. 60, no. 2, pp. 91-110, 2004.

[2] M. A. Fischler and R. C. Bolles, "Random sample consensus," *Commun. ACM*, vol. 24, no. 6, pp. 381-395, 1981.

[3] I. Amerini et al., "A SIFT-based forensic method for copy-move attack detection and transformation recovery," *IEEE TIFS*, vol. 6, no. 3, 2011.

[4] D. Tralic et al., "CoMoFoD - New database for copy-move forgery detection," *Proc. ELMAR*, 2013.

[5] V. Christlein et al., "An evaluation of popular copy-move forgery detection approaches," *IEEE TIFS*, vol. 7, no. 6, 2012.

[6] R. Arandjelovic and A. Zisserman, "Three things everyone should know to improve object retrieval," *CVPR*, 2012.

[7] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal*, 2000; Streamlit documentation, https://docs.streamlit.io